Lecture 08

Exceptions

Exceptions are events that occur during program execution that disrupt the normal flow of instructions. In Python, exceptions provide a powerful mechanism to handle errors and other exceptional events gracefully, making your code more robust and maintainable.

Contents

Basic Error Handling

Using `try` and `catch`

syntax

```
try:
    statement that might cause an error condition
except <<Error>>:
    Statements to recover from error.
```

The `try-except` block is the fundation of exception handling in Python. It allows you to attempt risky operations and handle potential errors gracefully.

```
#Basic Excpetion Handling in Python
# This script demonstrates how to handle exceptions in Python using try-
except blocks.
# It includes examples of catching specific exceptions and using a generic
exception handler.

try:
    # Attempt to open a file that does not exist
    with open('non_existent_file.txt', 'r') as file:
        content = file.read()
except FileNotFoundError as e:
    # Handle the specific exception for file not found
    print(f'Error: {e}. The file does not exist.')
```

Lecture 08

Exception Object

Whenever an exception occurs, Python creates an Exception object containing useful information about the erros. You can access this object using the as keyword so you are able to better handle the condition.

The exception object provides:

- **Type**: The class of the exception
- **Message**: A human-readable description
- **Arguments**: The arguments passed to the exception constructor

```
# accessing the excpetion object
try:
    result = 10 / 0
except ZeroDivisionError as e:    # 'e' is the exception object
    print('     Type:', type(e))
    print('  Message:', e)
    print('Arguments:', e.args)
```

Catching multiple exception

There are two approaches to handling multuple exception types:

option 1

```
try:
    statement that might cause an error condition
except (<<Error1>>, <<>>Error2):
    Statements to recover from error.
```

option2

```
try:
    statement that might cause an error condition
except <<Error1>>:
    Statements to recover from error.
except <<Error2>>:
    Statements to recover from error.
```

The second syntax is preferred because you are able to handle each error seperately.

```
#multiple exceptions
try:
    # Attempt to convert a string to an integer
    number = int('not_a_number')
except (ValueError, TypeError) as e:
    # Handle both ValueError and TypeError
    print(f'Error: {e}. Invalid conversion to integer.')
#alternate way to handle multiple exceptions
try:
    # Attempt to convert a string to an integer
    number = int('not_a_number')
```

```
except ValueError as e:
    # Handle ValueError
    print(f'ValueError: {e}. Invalid conversion to integer.')
except TypeError as e:
    # Handle TypeError
    print(f'TypeError: {e}. Invalid type for conversion to integer.')
```

The else Clause

The `else` clause executes **only if no exceptions were raised** in the `try` block. It's useful for code that should run only when the risky operation succeeds.

```
try:
    with open('data.txt', 'r') as file:
        content = file.read()
except FileNotFoundError as e:
    print(f'Error: {e}. The file does not exist.')
else:
    print('File read successfully.')
    print(f'Content length: {len(content)} characters')
```

The finally Clause

The `finally` clause always executes, regardless of whether an exception was raised or not. It's ideal for cleanup operations like closing files, releasing locks, or closing network connections

```
file = 'person.txt'

try:
    with open(file, 'r') as f:
        content = f.read()
except FileNotFoundError as e:
    print(f'Error: {e}. The file does not exist.')
else:
    print('File read successfully.')
finally:
    print('Execution completed, whether or not an error occurred.')
```

Raising Exception

You can manually trigger exceptions using the `raise` keyword.create and raise your own exception with `raise`

```
name = 'Narendra'

# if name == 'Narendra':
#     raise ValueError('Exception: Narendra is evil!')

try:
    if name == 'Narendra':
        raise ValueError('Exception: Narendra is evil!')
except ValueError as e:
```

```
    print('Exception:', e)


# of course, you can raise any exception you want, including custom
exceptions
```

Re-raising Exceptions

You can catch an exception, perform some action, and then re-raise it:

```
try:
    some_risky_operation()
except Exception as e:
    log_error(e)  # Log the error
    raise         # Re-raise the same exception
```

Custom Exception

Creating custom exceptions makes your code more expressive and easier to debug. Custom exceptions should inherit from the `Exception` class or a more specific built-in exception

(This can be skipped)

```
# Creating your own exception type
class MyError(Exception):
    pass

try:
    raise MyError("Something custom went wrong")
except MyError as e:
    print('Exception:', e)
```

Common Built-in Exceptions

Exception	Description
ValueError	Invalid value for an operation
TypeError	Operation on inappropriate type
KeyError	Dictionary key not found
IndexError	Sequence index out of range
FileNotFoundError	File or directory not found
ZeroDivisionError	Division by zero

Exception	Description
AttributeError	Attribute not found
ImportError	Module import failed

Best practice

- Catch only what you expect.
- Use finally for cleanup.
- Log errors instead on only printing.
- Gracefully handle user input.
- Handle external resources safely (like APIs).
- Raise exception with useful messages.
- Use custom exception for clarity.
- Don't abuse exception for flow control.

Summary

- Use specific exceptions instead of a blanket `except:` (which catches everything and can hide bugs).
- The `else` clause runs only when no exception occurs.
- The `finally` always runs, making it perfect for cleanup.
- Use `raise` to trigger exceptions intentionally with meaningful messages.
- **Custom exceptions** improve code clarity and make error handling more semantic.
- **Log errors** properly instead of just printing them
- **Avoid using exceptions** for normal program flow - they're for exceptional situations
- **Handle external resources** (files, network, databases) with proper exception handling
- **Validate user input** gracefully and provide clear feedback

Exception handling is not just about preventing crashes-it's about making your code resilient, maintainable, and user-friendly.

Lecture 08

File Input and Output

When a file is open, Python returns a **file object** that allows you to read or write to the file on disk.

File Modes

File mode determines what operations can be done on the file object

Character Meaning

- `r` open for reading (default)
- `t` text mode (default)
- `w` open for writing, truncating the file first
- `x` create a new file and open it for writing (fails if the file exist).
- `a` open for writing, appending to the end of the file if it exists
- `b` binary mode
- `+` open a disk file for updating (reading and writing)

Default mode: `'rt'` (reading text files with **UTF-8** encoding).

File Object Attribute

Before running the code below, ensure that there is a file with the correct name and in the correct folder.
Alternately, you can change the pathname of an existing file.

```
f = open('Data/person.txt', 'wt')
print(f'    name: {f.name}')        # pathname
print(f'    mode: {f.mode}')        # opening mode
```

```
print(f'encoding: {f.encoding}')     # encoding
print(f'  closed: {f.closed}')       # is file closed
print(f'  fileno: {f.fileno()}')     # file descriptor number
print(f'    tell: {f.tell()}')       # current position in file

f.close()
print(f'  closed: {f.closed}')
```

```
    name: Data/person.txt
    mode: wt
encoding: utf-8
  closed: False
  fileno: 3
    tell: 0
  closed: True
```

```
f = open('Data/vadim.txt', 'wt')
f.write('Hello Vadim\n')
f.write('-----------\n')
factor = 10
for x in range(12):
    f.write(f'{x:>3} x {factor} = {x * factor}\n')
f.write('end of file\n')
f.close()
f = open('data/ben.txt', 'wt')
data = 'Mary had a little lamb, its fleece was as white as snow'.split()
data = [word + '\n' for word in data]
f.writelines(data)
f.close()
```

Basic file writing

Writing data or information to a file is the only way to save information permanently.

Writing a single string to a file

The `write()` method of the file class is used to write a string to the associated file.

```
# Create and write to a new file (overwrites if exists)
with open('Data/person.txt', 'wt') as f:
    f.write('Created on jan-18-2025 by Narendra\n')
    f.write('12 times table\n')
    f.write('--------------\n')
    for x in range(1, 13):
        f.write(f'{x} x 12 = {x * 12}\n')
```

Note: Using the `with` statement ensure the files is automatically closed, even if an error occurs.

Appending to an Existing File

If the file is open in append mode, then write puts text at the end of the existing file

```
with open('Data/person.txt', 'at') as f:
    f.write('this should be at the bottom of the file')
```

Writing Multiple Lines

The `writelines()` method writes a sequence of strings to the file

```
data = ['Created on jan-18-2025 by Narendra\n',
        '12 times table\n',
        '--------------\n']

table = [f'{x:>3} x {12} = {x * 12} \n' for x in range(1, 13) ]

data.extend(table)
# notice that all the items in the list are string that ends with '\n'
# so we can use writelines() to write all the lines at once
f = open('Data/table.txt', 'wt')
f.writelines(data)
f.close()
# f = open('person.dat', 'wb')
# f.write(b'Created on jan-18-2025 by Narendra\n')
# f.write(b'12 times table\n')
# f.write(b'--------------\n')
# for x in range(1, 13):
#     line = f'{x} x 12 = {x * 12}'
#     f.write(bytearray(line, encoding ='utf-8'))
#     f.write(b'\n')
# f.close()
```

Reading Methods

Method	Description
`read(size=-1)`	Reads the entire file or specified number of characters
`readline(size=-1)`	Reads one line, or the specified number of lines
`readlines(hint=-1)`	Reads all lines into a list
`tell()`	Returns the current stream position in file
`seek(position)`	Move to specified position in file

Reading the Entire File

```
with open('Data/person.txt') as f:
    print(f'      first read: {f.read(10)}')            # read first 10
characters
    print(f'current position: {f.tell()}')              # current position in
the file
    print(f'     the balance: {f.read()}')
    print('\n\nresetting to the start of the file')
    f.seek(0)                                          # move back to the
start of the file
```

```
    print(f.read())                                          # read the entire
file
```

Reading Line by Line

```
with open('Data/person.txt') as f:
    print(f'First line: {f.readline()}')
```

Reading All Lines into a List

```
with open('Data/person.txt') as f:
    all_lines = f.readlines()
    print(f'All lines: {all_lines}')
```

Recommended: Iterating Over File Lines

The most efficient and Pythonic way to read a file line by line:

```
with open('Data/person.txt') as f:
    for line in f:
        print(line.strip())  # strip() removes the trailing newline character

# with this technique, you don't need to explicitly close the file
# it will be closed automatically when you exit the with block
```

Benefits:

- Memory efficient (doesn't load entire file)
- Automatic file closure
- Clean, readable code

Working with Binary Files

Binary files are more compact and can be a slight deterrent to prying eyes.
Only binary data can be written to a file.
Reading will give binary data.

Writing binary data

```
data = bytes([0, 1, 2, 3, 4, 5])

with open('Data/binary.bin', 'wb') as file:
    # file.write('this is some plain text\n')           #this does not write
correctly
    file.write(b'this is some binary text\n')

    # Encode string to bytes
    file.write('this is some more binary text!\n'.encode('utf-8'))

    # Write bytes array
    file.write(data)
```

Reading binary data

```python
with open('Data/binary.bin', 'rb') as file:
    binary_data = file.read()
    print(binary_data)
with open('person.txt') as f:
    print('file contents:')
    print(f'{f.readline()}')
    print(f'{f.read()}')
```

Using Different Encodings

In Python, encoding in file writing refers to the way text characters are converted into bytes before being stored in a file.

When you write to a file in text mode, Python takes your string (which internally uses Unicode) and encodes it into a sequence of bytes according to the specified character encoding scheme — like UTF-8, UTF-16, ASCII, etc.

Why encoding matters

Computers store files as bytes, but human-readable text is made of characters.
Different encodings map characters to bytes differently.
If you choose the wrong encoding when writing, the file might not display correctly when read later.

Common Encodings

Encoding	Description	Use Case
UTF-8	Universal, supports all Unicode characters	Recommended default
ASCII	Limited basic English chracters	Legacy systems
UTF-16/32	Wider byte representation	Specific compatibility needs
Latin-1	Western European characters	Legacy European systems

Suggestion:

When you write a file with a specific encoding, read it later with the same encoding.
UTF-8 is almost always the safest choice.

Writing and Reading with specific encoding

```python
with open('Data/utf8_example.txt', 'w', encoding='utf-8') as file:
    file.write("Some text with special characters: ñ, é, ü\n")
```

```
# Reading with specific encoding
with open('Data/utf8_example.txt', 'r', encoding='utf-8') as file:
    print(file.read())
#another example
with open('Data/utf8_example.txt', 'w', encoding='utf-8') as file:
    file.write("Hello, world! — こんにちは — Привет")

# Reading with specific encoding
with open('Data/utf8_example.txt', 'r', encoding='utf-8') as file:
    print(file.read())
```

1. working with CSV Files

CSV files, short for Comma-Separated Values files, are a simple and widely-used format for storing tabular data—like what you'd see in a spreadsheet.
What is a CSV file?
A CSV file is a plain text file where:

- Each line represents a row of data.
- Each value in the row is separated by a comma (or sometimes another delimiter like a semicolon or tab).

Why are CSV files important?

Here are some key reasons:

1. Simplicity & Universality
   CSV files are easy to create, read and edit. Almost every programming language and data tool (like Excel, Google Sheets, databases and programming languages like Python, R, etc.) supports them.

2. Lightweight Format
   Since they're plain text, CSV files are small in size and quick to load, process or transfer.

3. Easy Data Exchange
   They're ideal for sharing data between different systems or platforms—especially when exporting or importing data from databases, spreadsheets, or web applications.

4. Human-Readable
   You can open a CSV file in any text editor and understand the data without needing special software.

5. Automation-Friendly
   CSVs are often used in data pipelines, machine learning workflows, and automated reporting systems because they're easy to parse and manipulate programmatically.

6. Integration with analytics tools
   Many data analysis tools (Pandas, R, Tableau, Power BI) directly read CSVs.

Lecture 08

Limitations of CSV Files

- No support for formatting (colors, formulas, multiple sheets like Excel)
- Can't handle complex data relationships (like a database)
- Risk of errors if data contains commas or line breaks

Writing to a CSV File

```
import csv

data = [
    ['Name', 'Age', 'City'],
    ['Alice', 30, 'Toronto'],
    ['Bob', 25, 'Vancouver']
]

with open('Data/people.csv', 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerows(data)
```

Reading from a CSV File

```
with open('Data/people.csv', 'r', newline='') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Working with Dictionaries

```
# Writing dictionaries
data = [
    {'name': 'Alice', 'age': 30, 'city': 'Toronto'},
    {'name': 'Bob', 'age': 25, 'city': 'Vancouver'}
]

with open('Data/people_dict.csv', 'w', newline='') as file:
    fieldnames = ['name', 'age', 'city']
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()  # Write column headers
    writer.writerows(data)

# Reading as dictionaries
with open('Data/people_dict.csv', 'r', newline='') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)  # Each row is a dictionary
```

Working with json files

JSON (JavaScript Object Notation) is a lightweight format for storing structured data, widely used for configuration files and data exchange.

Why Use JSON?

- **Structured data**: Supports nested objects and arrays
- **Data types**: Preserves strings, numbers, booleans, null
- **Human-readable**: Easy to read and edit
- **Web standard**: Native format for web APIs
- **Language-independent**: Supported by virtually all programming languages

Writing to a JSON File

```
import json

data = {
    'name': 'Ilia Nika',
    'age': 30,
    'city': 'Toronto',
    'skills': ['Python', 'JavaScript', 'SQL'],
    'active': True
}

with open('Data/data.json', 'w') as file:
    json.dump(data, file, indent=4)
```

Reading a JSON files

```
with open('Data/data.json', 'r') as file:
    data = json.load(file)
    print(data)
```

Other file I/O

1. Processing excel, xml and sqlite files
2. Reading and writing parqet files

Summary

File I/O is fundamental to programming, enabling data persistence and exchange between applications. Mastering these techniques is essential for building robust Python applications.

- Use Context Managers: Always use `with` statements for automatic file closure
- Choose Correct Modes: Select appropriate file modes (`r`, `w`, `a`, `rb`, etc.)
- Text files use encoding to convert between strings and bytes
- Specify Encodings: Always specify encoding (UTF-8 recommended) for consistency
- Handle Exceptions: Implement proper error handling for file operations
- Process Efficiently: For large files, read line by line instead of loading entire file
- Binary files work directly with bytes for efficiency
- Use Standard Libraries: Leverage csv and json modules for structured data
- CSV files provide a universal format for tabular data
- JSON files handle structured data with data type preservation

Lecture 08

CSV files are essential because they provide a universal, lightweight, and easy way to store and exchange structured data across different platforms. Whether you're a programmer, data analyst, or business user, CSVs make data handling simple and efficient.

Key Takeaway

Choose the right file format and operations for your use case:

- Plain text: Simple data, logs
- CSV: Tabular data, spreadsheets
- JSON: Structured data, configurations
- Binary: Performance-critical or non-text data